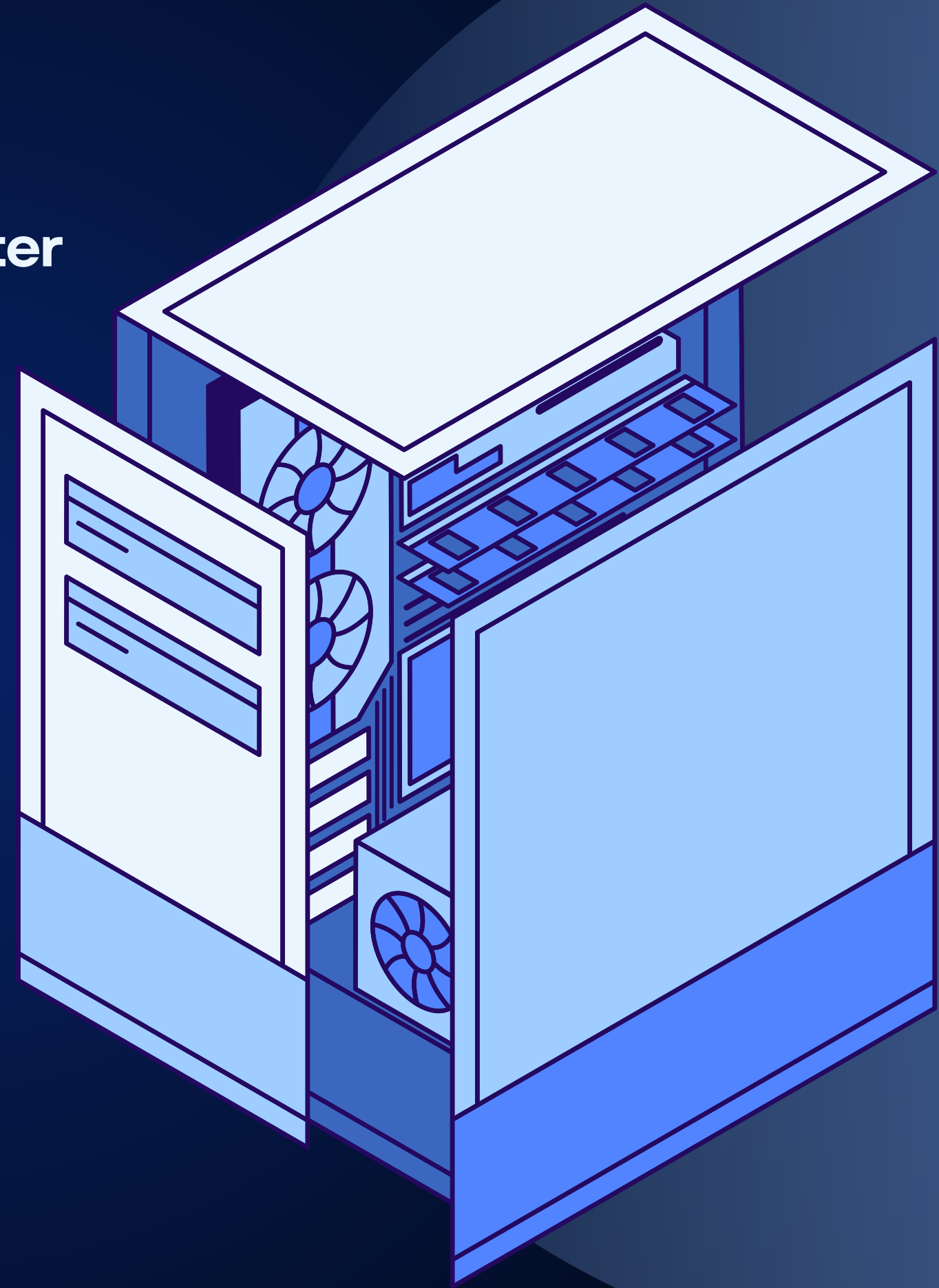


METODE JACOBI



Kelompok 7 - A12





MEMBERS OF THE GROUP

Nabila Salsabil

230101501025

Fathur Rahim

230101501029

Fitri Rachmadani

230101501030



Kelompok 7-A12



METODE JACOBI

→ Carl Gustav Jacob Jacobi (Jerman, 1804–1851)

DEFINISI

Metode Iterasi Jacobi merupakan algoritma dalam analisis numerik yang digunakan untuk menentukan solusi hampiran dari Sistem Persamaan Linear (SPL). Berbeda dengan metode eliminasi langsung (seperti Gauss) yang bersifat finit, metode ini bekerja melalui skema perbaikan nilai secara berulang (iteratif).

ALUR KERJA

Menentukan vektor tebakan awal



Memisahkan setiap variabel dominan dari masing-masing baris persamaan.



Menghitung nilai variabel baru secara serentak berdasarkan data dari iterasi sebelumnya.



Mengevaluasi apakah hasil pendekatan sudah mencapai konvergensi yang diinginkan.

MEKANISME METODE JACOBI



Transformasi Matematis

Mengubah sistem $Ax = b$ menjadi bentuk iteratif:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$



Nilai variabel baru (k+1) tidak dipengaruhi oleh hasil variabel lain dalam siklus iterasi yang sama.

Konvergensi

Matriks harus Dominan Secara Diagonal agar solusi tidak menjauh (divergen).

KRITERIA BERHENTI

- Toleransi Galat: Berhenti jika selisih nilai antar-iterasi sudah sangat kecil
- Iterasi Maksimum: Batas pengulangan untuk mencegah infinite loop pada sistem yang tidak stabil.

Contoh Soal

$$\begin{aligned}10x_1 - x_2 + 2x_3 + 0x_4 &= 6 \\ -x_1 + 11x_2 - x_3 + 3x_4 &= 25 \\ 2x_1 - x_2 + 10x_3 - x_4 &= -11 \\ 0x_1 + 3x_2 - x_3 + 8x_4 &= 15\end{aligned}$$



$$x_1 = \frac{(6 + x_2 - 2x_3)}{10}$$

$$x_2 = \frac{(25 + x_1 + x_3 - 3x_4)}{11}$$

$$x_3 = \frac{(-11 - 2x_1 + x_2 + x_4)}{10}$$

$$x_4 = \frac{(15 - 3x_2 + x_3)}{8}$$

Tebakan Awal

$$x_1^0 = 0$$

$$x_2^0 = 0$$

$$x_3^0 = 0$$

$$x_4^0 = 0$$

Iterasi Ke1

$$x_1 = \frac{(6 + 0 - 2(0))}{10} = 0.600$$

$$x_2 = \frac{(25 + 0 + 0 - 3(0))}{11} = 2.273$$

$$x_3 = \frac{(-11 - 2(0) + 0 + 0)}{10} = -1.100$$

$$x_4 = \frac{(15 - 3(0) + (0))}{8} = 1.875$$



Iterasi Ke 2

$$\begin{aligned}x_1'' &= \frac{(6 + 2.273 - 2(-1.100))}{10} \\&= \frac{(6 + 2.273 + 2.200)}{10} \\&= 1.047\end{aligned}$$

$$\begin{aligned}x_2'' &= \frac{(25 + 0.600 - 1.100 - 3(1.875))}{11} \\&= \frac{(25 + 0.600 - 1.100 - 5.625)}{11} \\&= 1.716\end{aligned}$$

$$\begin{aligned}x_3'' &= \frac{(-11 - 2(0.600) + 2.273 + 1.875)}{10} \\&= \frac{(-11 - 1.200 + 2.273 + 1.875)}{10} \\&= -0.805\end{aligned}$$

$$\begin{aligned}x_4'' &= \frac{(15 - 3(2.273) + (-1.100))}{8} \\&= \frac{(15 - 6.819 - 1.100)}{8} \\&= 0.885\end{aligned}$$

Iterasi Ke 3

$$\begin{aligned}
 x_1^3 &= \frac{(6 + 1.716 - 2(-0.805))}{10} \\
 &= \frac{(6 + 1.716 + 1.610)}{10} \\
 &= 0.933
 \end{aligned}$$

$$\begin{aligned}
 x_2^3 &= \frac{(25 + 1.047 - 0.805 - 3(0.885))}{11} \\
 &= \frac{(25 + 1.047 - 0.805 - 2.655)}{11} \\
 &= 2.053
 \end{aligned}$$

$$\begin{aligned}
 x_3^3 &= \frac{(-11 - 2(1.047) + 1.716 + 0.885)}{10} \\
 &= \frac{(-11 - 2.094 + 1.716 + 0.885)}{10} \\
 &= -1.049
 \end{aligned}$$

$$\begin{aligned}
 x_4^3 &= \frac{(15 - 3(1.716) + (-0.805))}{8} \\
 &= \frac{(15 - 5.148 - 0.805)}{8} \\
 &= 1.131
 \end{aligned}$$

$$\begin{aligned}
 \epsilon_{x1} &= \frac{(0.933 - 1.047)}{0.933} \times 100 \\
 &= 12.2\%
 \end{aligned}$$

$$\begin{aligned}
 \epsilon_{x2} &= \frac{(2.053 - 1.716)}{2.053} \times 100 \\
 &= 16.4\%
 \end{aligned}$$

$$\begin{aligned}
 \epsilon_{x3} &= \frac{(-1.049 - (-0.805))}{(-1.049)} \times 100 \\
 &= 23.3\%
 \end{aligned}$$

$$\begin{aligned}
 \epsilon_{x4} &= \frac{(1.131 - 0.885)}{1.131} \times 100 \\
 &= 21.8\%
 \end{aligned}$$

Class Jacobi



Class Jacobi digunakan untuk menyelesaikan SPL menggunakan metode iteratif Jacobi.

Karakteristik utama:

- 1. Menggunakan nilai iterasi sebelumnya**
- 2. Semua variabel diperbarui secara simultan**
- 3. Proses iterasi dilakukan sampai memenuhi toleransi error**





Atribut utama:

Self.A → matriks koefisien

Self.b → vektor konstanta

x0 → tebakan awal solusi

Self.toleransi → batas toleransi error

max_iter → jumlah maksimum iterasi

Struktur Atribut

Atribut tambahan:

history → menyimpan hasil setiap iterasi

errors → menyimpan nilai error

iterations → jumlah iterasi

converged → status konvergensi

`_init_()` Constructor yang otomatis dijalankan saat objek dibuat. Berfungsi untuk menginisialisasi data awal seperti matriks, vektor, tebakan awal, dan parameter iterasi.

`cek_diagonal_dominan()`

Digunakan untuk memeriksa apakah matriks memenuhi syarat diagonal dominan, yang berpengaruh pada konvergensi metode Jacobi.

`solve()`

Method utama yang menjalankan proses iterasi Jacobi:

Menghitung nilai variabel baru

Menggunakan nilai iterasi sebelumnya

Menghitung error (norm tak hingga)

Mengecek kondisi konvergensi

Menghentikan iterasi jika $\text{error} < \text{toleransi}$

tampilkan_iterasi()

Menampilkan seluruh hasil iterasi dalam bentuk tabel, termasuk nilai variabel dan error tiap langkah.

hasil()

Menampilkan solusi akhir dari sistem persamaan, jumlah iterasi, error akhir, dan status konvergensi.

residu()

Menghitung nilai residu: $Ax - b$

Implementasi Jacobi menerapkan prinsip OOP sebagai berikut:

- ◆ **Encapsulation**

Semua data (matriks, vektor, parameter iterasi) dan fungsi dikemas dalam satu class sehingga lebih terorganisir.

- ◆ **Abstraction**

Pengguna hanya perlu memanggil method seperti `solve()` tanpa memahami detail perhitungan di dalamnya.

- ◆ **Modularity**

Struktur class memudahkan pengembangan fitur tambahan seperti logging iterasi, error tracking, atau visualisasi.

1. Import Library:

```
from Metode_Iteratif.jacobi import Jacobi
from Metode_Iteratif.gauss_seidel import GaussSeidel
import numpy as np
import matplotlib.pyplot as plt
```

Memanggil modul dan library (NumPy, Matplotlib).

2. `__init__` (Constructor):

```
def __init__(self, A, b, x0=None, tol=1e-6, max_iter=100):
```

Menyiapkan hal hal yang diperlukan seperti matriks A, vektor b, tebakan awal, dan batas error.

3. solve():

```
def solve(self):
```

Mesin utama yang menghitung variabel secara berulang sampai hasilnya akurat.

4. tampilkan_iterasi():

```
def tampilkan_iterasi(self):
```

Menyusun semua riwayat hitungan ke dalam tabel supaya prosesnya terlihat jelas.

5. Hasil():

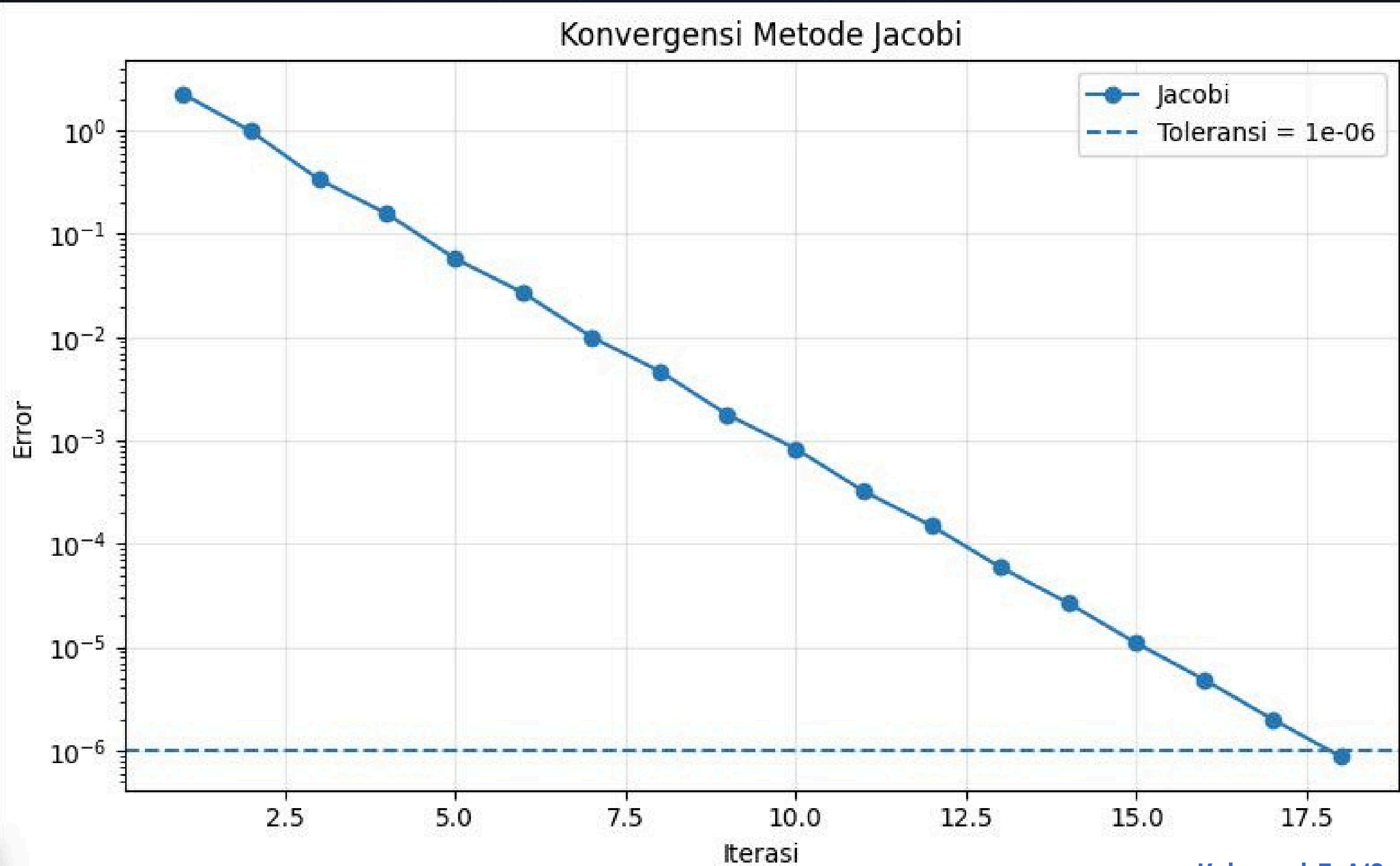
```
def hasil(self):
```

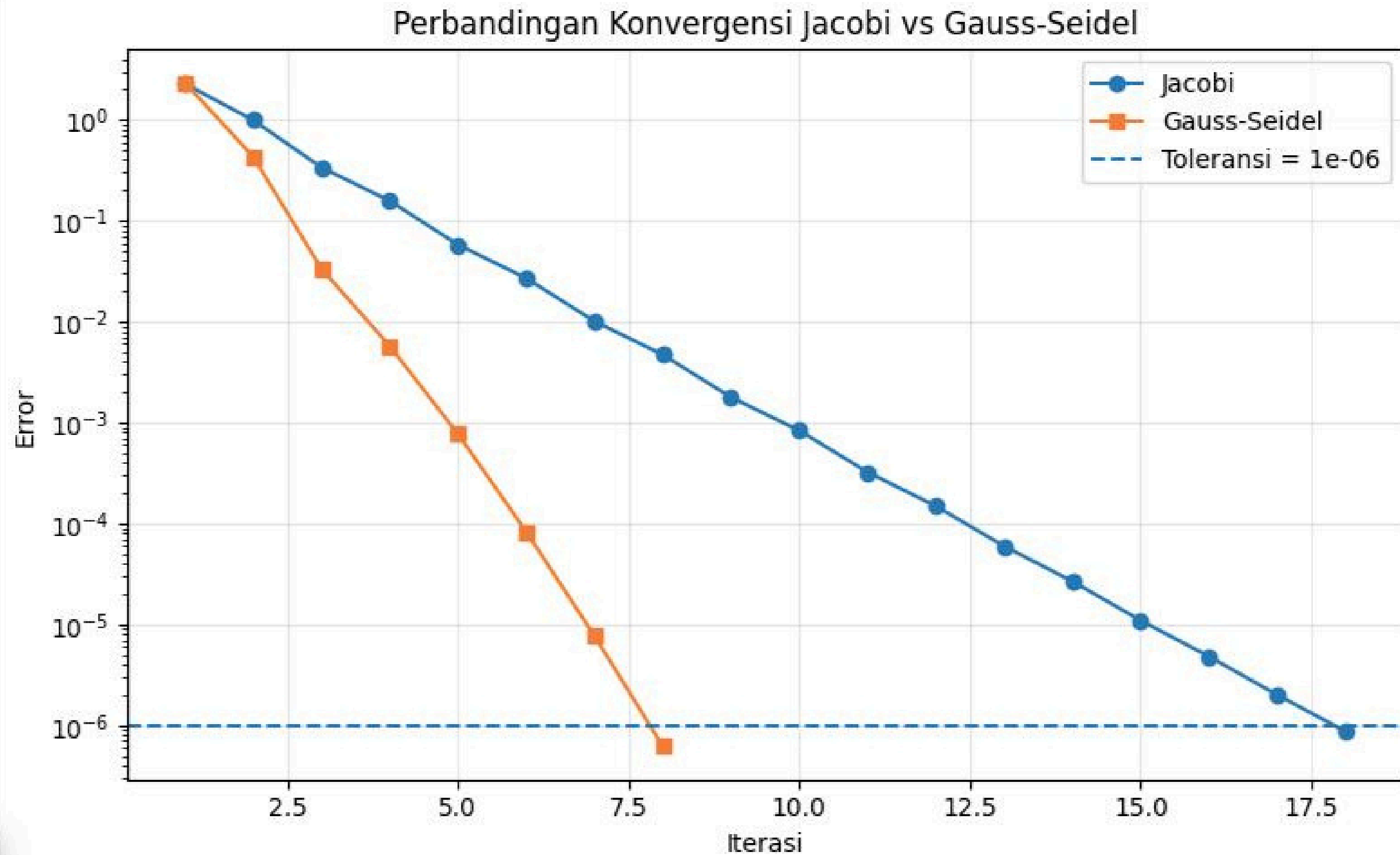
Menampilkan jawaban akhir, jumlah iterasi yang dipakai, dan status keberhasilannya.

6. residu():

```
def residu(self):
```

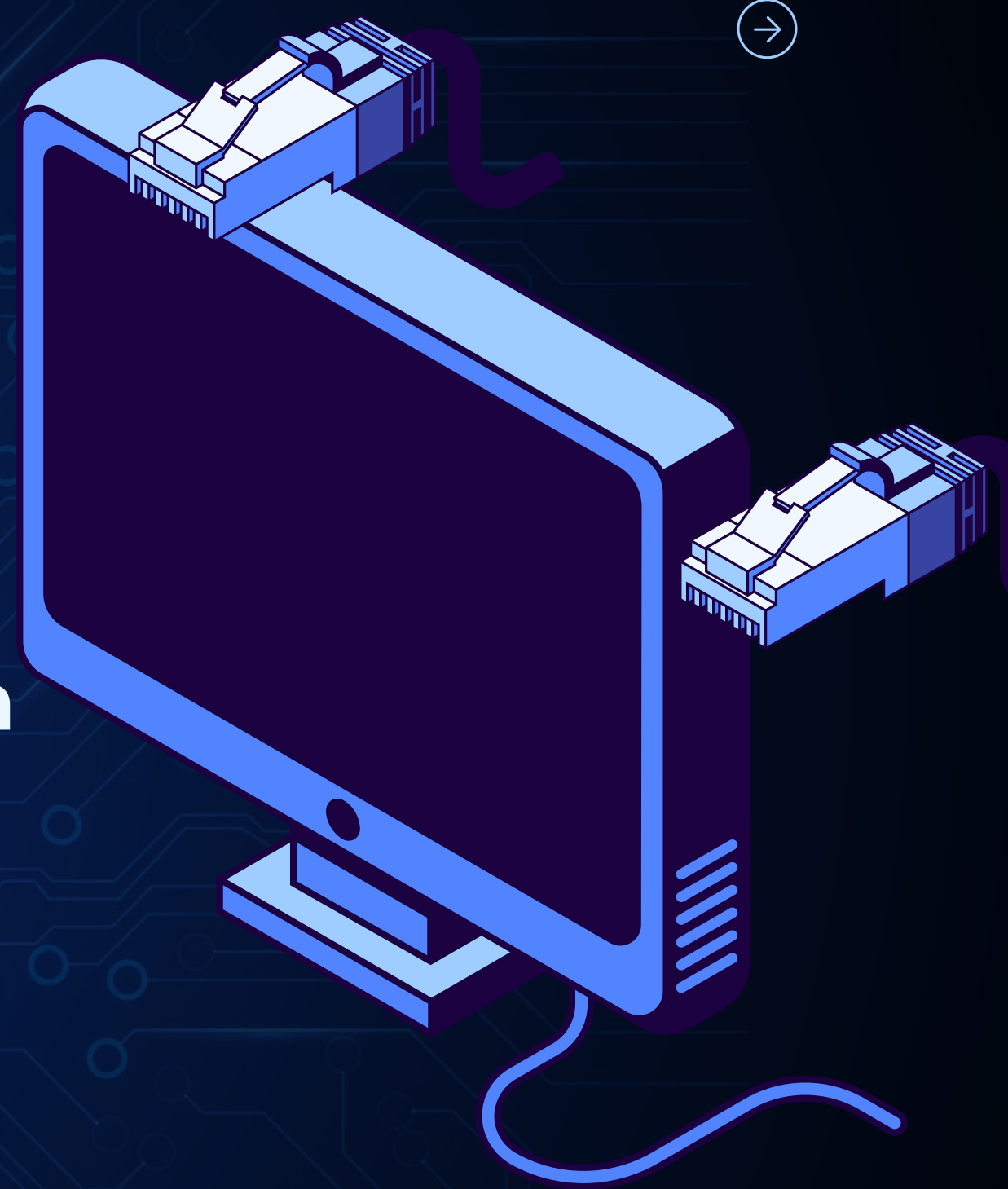
Pengecekan terakhir untuk memastikan jawaban sangat mendekati nilai nol (validasi).





KESIMPULAN

Metode Jacobi adalah alat yang sangat kuat untuk menyelesaikan sistem persamaan linear besar yang sering kita temukan dalam pemodelan matematika. Dengan bantuan pemrograman, proses iterasi yang membosankan jika dihitung manual bisa diselesaikan dalam hitungan detik oleh komputer.



Dokumentasi





THANK
YOU.